# Securitize Vault V1 Audit Report

Prepared by Cyfrin

Version 2.0


**Lead Auditors**

Hans


March 10, 2026

# Contents


**1 About Cyfrin** **2**

**2 Disclaimer** **2**

**3 Risk Classification** **2**

**4 Protocol Summary** **2**
- 4.0.1 Actors and Roles . . . 2
- 4.0.2 Deposit/Redeem Flow . . . 3
- 4.0.3 Liquidation Flow . . . 3

**5 Audit Scope** **3**

**6 Executive Summary** **3**

**7 Findings** **5**
- 7.1 Critical Risk . . . 5
  - 7.1.1 Incomplete access control over deposit and redeem . . . 5
- 7.2 Low Risk . . . 6
  - 7.2.1 Unsafe ERC20 Operations should not be used . . . 6
- 7.3 Informational . . . 7
  - 7.3.1 Some function names are misleading . . . 7
  - 7.3.2 Some comments are misleading . . . 7
  - 7.3.3 Avoid emitting unnecessary events . . . 7
  - 7.3.4 General notes on the design . . . 7
- 7.4 Gas Optimization . . . 9
  - 7.4.1 Unnecessary modifier . . . 9

# 1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

# 2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

# 3 Risk Classification

| | **Impact: High** | **Impact: Medium** | **Impact: Low** |
|---|---|---|---|
| **Likelihood: High** | Critical | High | Medium |
| **Likelihood: Medium** | High | Medium | Low |
| **Likelihood: Low** | Medium | Low | Low |

# 4 Protocol Summary

Securitize and its subsidiaries have a global footprint with offices worldwide, giving them a unique perspective and the opportunity to lead the evolution of digital asset securities (security tokens) in and across multiple markets. `SecuritizeVault` is designed to introduce a way for users to participate in other Defi projects and earn rewards. Users can deposit asset tokens into the vault and receive a token representing their share of the vault. The SecuritizeVault contract is an upgradeable smart contract that implements an asset vault with roles for redeemers and liquidators, facilitating deposit, redemption, and liquidation.

#### 4.0.1 Actors and Roles

1. **Actors**:
   - **Users**: General users who deposit assets and get shares from the vault.
   - **Owner**: An account that can upgrade the contract and also pause/resume the contract.
   - **Admin**: An account with the `DEFAULT_ADMIN_ROLE`, responsible for managing redeemers and liquidators, and making administrative changes.
   - **Redeemers**: Accounts with the `REDEEMER_ROLE`, authorized to redeem shares from the vault.
   - **Liquidators**: Accounts with the `LIQUIDATOR_ROLE`, authorized to liquidate shares.
2. **Roles**:
   - **Owner**: Can upgrade and pause/resume the contract.
   - **Default Admin Role**: Can assign and revoke `REDEEMER_ROLE` and `LIQUIDATOR_ROLE`, change admin, and manage liquidation settings.
   - **Redeemer Role**: Can redeem shares for assets.
   - **Liquidator Role**: Can liquidate shares for the specified liquidation token, even if the liquidation is not open to the public.

### 4.0.2 Deposit/Redeem Flow

The deposit and redeem flows are the core functionalities of the SecuritizeVault contract, allowing users to deposit assets and redeem shares for assets, respectively. The protocol enforces a 1:1 ratio between the deposited assets and the shares minted. Users can deposit asset tokens into the vault in exchange for shares that represent their stake in the vault. These shares are minted based on the amount of assets deposited. Redeemers can redeem their shares to withdraw assets from the vault. The shares are burned in exchange for the underlying assets.

### 4.0.3 Liquidation Flow

Liquidators can convert shares into a specified liquidation token. Authorized liquidators (or public users if allowed) can call the liquidation function, which transfers the shares to the redemption contract. The redemption contract returns liquidation token, which is returned to the liquidator.

# 5 Audit Scope

All files in the `contracts` directory except the `mock` subfolder are in the scope of the audit.

# 6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the Securitize Vault V1 smart contracts provided by Securitize. In this period, a total of 7 issues were found.

**Summary**

| | |
|---|---|
| Project Name | Securitize Vault V1 |
| Repository | bc-securitize-vault-sc |
| Commit | 036e31371a9ca0afd648ac7d7b78bd59a44bb1ee12... |
| Audit Timeline | Aug 2nd - Aug 5th, 2024 |
| Methods | Manual Review |

**Issues Found**

| | |
|---|---|
| Critical Risk | 1 |
| High Risk | 0 |
| Medium Risk | 0 |
| Low Risk | 1 |
| Informational | 4 |
| Gas Optimizations | 1 |
| Total Issues | 7 |

**Summary of Findings**

| | |
|---|---|
| [C-1] Incomplete access control over deposit and redeem | Resolved |
| [L-1] Unsafe ERC20 Operations should not be used | Resolved |
| [I-1] Some function names are misleading | Resolved |
| [I-2] Some comments are misleading | Resolved |
| [I-3] Avoid emitting unnecessary events | Resolved |
| [I-4] General notes on the design | Acknowledged |
| [G-1] Unnecessary modifier | Resolved |

# 7 Findings

## 7.1 Critical Risk

### 7.1.1 Incomplete access control over deposit and redeem

**Description:** `SecuritizeVault` was designed to limit access to deposit and redeem but due to missed overrides it is still possible to use the access-controlled functions. The vault contract inheritied the OpenZeppelin's `ERC4626Upgradeable` and `ERC4626Upgradeable` has several public functions exposed by default. Especially, `ERC4626Upgradeable` allows mint/deposit and redeem/withdraw in two ways.

```
function deposit(uint256 assets, address receiver) public virtual returns (uint256);
function mint(uint256 shares, address receiver) public virtual returns (uint256);

function withdraw(uint256 assets, address receiver, address owner) public virtual returns (uint256);
function redeem(uint256 shares, address receiver, address owner) public virtual returns (uint256);
```

Note that `deposit()` and `withdraw()` functions accept the asset amount while `mint()` and `redeem()` functions accept the share amount as parameters.

`SecuritizeVault` has overriden the function `deposit()` and `redeem()` with additional access controls. For the function `deposit()`, the vault only allowed depositing to itself and for the function `redeem()` the vault allowed only redeemers. But because the vault did not override the other functions `mint()` and `withdraw()`, it is still possible to work around this limitation.

Moreover, deposit and redeem will be possible even when the vault is paused by the owner.

**Impact:** Access control is broken and anyone can deposit to any other address and anyone can redeem while it is not the protocol's intention. Furthermore, deposit and redeem will function even when the vault contract is paused by the owner. We evaluate the impact to be CRITICAL.

**Proof Of Concept:** Put the test inside `deposit.test.ts`.

```
it('Cyfrin: Can deposit using a mint function even if receiver is not the same as sender', async ()
↪  => {
  const { vault, dsMock, redeemer, owner } = await loadFixture(deployRedemptionVault);
  await dsMock.mint(owner.address, amount);
  await dsMock.approve(vault.target, amount);

  console.log(await vault.balanceOf(redeemer.address));
  await vault.mint(amount, redeemer.address);
  console.log(await vault.balanceOf(redeemer.address));
});
```

**Recommended Mitigation:**

- Override the other functions `mint()` and `withdraw()` with the same access control.
- Note that `deposit()` and `redeem()` is not an ideal pair to be implemented and exposed. In general, it is either deposit and withdraw, or mint and redeem.
- Note that the modifier `receiverSenderNotEqual(receiver)` is technically not meaningful because anyone can transfer the shares (vault token) to others.

**Securitize:** Fixed in Commit 52350aa.

**Cyfrin:** Verified.

- Public functions `mint()` and `withdraw()` are overridden to revert all the time.
- The modifier `receiverSenderNotEqual(receiver)` has been removed.

## 7.2 Low Risk

### 7.2.1 Unsafe ERC20 Operations should not be used

**Description:** In several places, the current implementation uses the function `transfer()` to transfer ERC20 tokens.

```
SecuritizeVault.sol
186: bool success = assetToken.transfer(_to, _transferAmount);
307: bool success = liquidationToken.transfer(msg.sender, assets);
```

But not all ERC20 tokens adhere to the standard. Some tokens do not return boolean and some tokens do not revert on failure.

**Recommended Mitigation:** Use OpenZeppelin's SafeERC20 where the safeTransfer and safeTransferFrom functions handle the return value check as well as non-standard-compliant tokens

**Securitize:** Fix in 3cd6413.

**Cyfrin:** Verified.

## 7.3 Informational

### 7.3.1 Some function names are misleading

**Description:** Some function names are misleading given the usages.

- The modifier `receiverSenderNotEqual()` is used to ensure the `msg.sender` is equal to the argument and the name that includes the parameter name is not desirable here, especially given that the same modifier is used to check if the `msg.sender==owner` at L273. `senderEqualTo` or `msgSenderEqualTo` would be better.

```
modifier receiverSenderNotEqual(address _receiver) {
        require(_receiver == msg.sender, "Receiver must be equal to sender");
        _;
    }
```

- `assetIsImpaired()` is better than `impairedAsset()` given that this function does not return the actual impaired amount but the status as boolean.
- `vaultIsImpaired()` is better than `impairedVault()`.

**Securitize:** Fixed in b337b12.

**Cyfrin:** Verified.

### 7.3.2 Some comments are misleading

**Description:** Some comments are wrong/misleading.

In the below snippet, `impairedAssetBalance` is not a flag, it was supposed to be the function `impairedAsset()`.

```
SecuritizeVault.sol
177: * - The `impairedAssetBalance` flag must be true, indicating that the asset balance is indeed
↪  impaired.
```

The comment in the below snippet is incorrect (L321) because there is no possibility that the vault contract transfers assets directly to users.

```
SecuritizeVault.sol
320:      * This can occur if users transfer assets directly to the vault instead of using the deposit
↪  function,
321:      * or if the vault transfers assets directly to users instead of using the redeem function.
```

**Securitize:** Fixed in a2bae8 and b337b1.

**Cyfrin:** Verified.

### 7.3.3 Avoid emitting unnecessary events

**Description:** The function `setLiquidationOpenToPublic` is used to set the state `liquidationOpenToPublic` and it is set to the provided parameter regardless of the current status. If the provided parameter is the same to the current value, a redundant event `LiquidationOpenToPublic` will be emitted.

**Securitize:** Fixed in a2bae86.

**Cyfrin:** Verified.

### 7.3.4 General notes on the design

**Description:**

1. In general, it is not clear why the team inherited ERC4626 vault. The main point of using an ERC-4626 vault is to provide a standardized interface for managing "yield-bearing" tokenized vaults, enabling consistent tracking and calculation of share prices (i.e., the value of vault shares relative to the underlying assets). But the

`SecuritizeVault` enforces 1:1 ratio between the share and the asset, it voids the main point. Furthermore, the vault has a function `liquidate()` that is not in the ERC4626 standard and it could confuse users and engineers because the term `liquidate` is used in lending protocols. The current implementation does not show any clear evidence of necessity of using ERC4626. It could be an overkill and also error prone.

2. The current implementaion has `impairedAssetBalance` and `impairedVaultBalance` and each of them has its relevant transfer functions. It is not clear why these are necessary. Based on the context given by the team during the call, we think these are introduced to swipe the redundant assets or shares that are sent to the vault contract "by mistake". In general, it is not a big concern for the protocol itself because it is after all the caller's mistake and that is not the protocol's responsibility. If the protocol is really concerned with the user mistakes, it is enough to have a single admin function. (refer to Compound's sweep function below.) Note that the current implementation also consumes unnecessary gas because the internal function `_validateBalance()` is called for every transaction.

```
/**
 * @notice A public function to sweep accidental ERC-20 transfers to this contract. Tokens are sent
 ↪  to admin (timelock)
 * @param token The address of the ERC-20 token to sweep
 */
function sweepToken(EIP20NonStandardInterface token) override external {
    require(msg.sender == admin, "CErc20::sweepToken: only admin can sweep tokens");
    require(address(token) != underlying, "CErc20::sweepToken: can not sweep underlying token");
    uint256 balance = token.balanceOf(address(this));
    token.transfer(admin, balance);
}
```

**Securitize:** We chose to implement the `ERC-4626` standard over a `wrapped ERC-20` due to our future plans to implement a variable Net Asset Value (NAV) price. This means that the share-to-asset ratio will vary, making `ERC-4626's` standardized interface for managing yield-bearing tokenized vaults beneficial. For the current use case, we are enforcing a 1:1 ratio, but this will change as we evolve our implementation.

Regarding the `liquidate()` function, it was included to meet the specific needs of our integration with lending protocols which will be the primary destination for the szTokens. The `liquidate()` method facilitates the process of converting `szTokens` to USDC. While we acknowledge that this function is not part of the `ERC-4626` standard and might cause some confusion, it is essential for our intended functionality. Additionally, not all users will be able to redeem their `szTokens` for a `DS Token`, so the only exit for them will be the liquidation method.

We removed the `_validateBalance()` and modified the impiarment methods:

```
/**
 * @dev Checks if the vault's asset balance impairment.
 *
 * @return uint256 Returns true if the asset balance is impaired, false otherwise.
 */
function assetImpairedBalance() public view returns (uint256) {
    uint256 assetBalance = IERC20(asset()).balanceOf(address(this));
    uint256 impairedAssetBalance;
    if (assetBalance > totalSupply()) {
        impairedAssetBalance = assetBalance - totalSupply();
    }
    return impairedAssetBalance;
}

/**
 * @dev Checks if the vault's balance imparement.
 *
 * @return uint256 Returns the impaired vault balance.
 */
function vaultImpairedBalance() public view returns (uint256) {
    uint256 impairedVaultBalance;
    if (balanceOf(address(this)) > 0) {
        impairedVaultBalance = balanceOf(address(this));
```

```
        }
        return impairedVaultBalance;
    }
```

Now this is a view, so the calculation will be on demand and gas free. The need to extract the wrongly deposited tokens, is because of the DS Token is a regulated token, and we need to have an error fix mechanism. When an error occurs we need to have the consistency 1:1 ratio. If a szToken is locked, then the DS Token will be also locked. Compound's sweep method as-is is not suitable, and also is very similar to the `transferImpairedXbalance` methods. For the current model, the vault will have a specific owner-admin, and the owner-admin will be able to fix the error on demand.

```
    /**
     * @dev Transfers the impaired asset balance to a specified address.
     * This method is used when the asset balance of the vault is considered impaired,
     * meaning it does not reflect the expected balance due to incorrect transfers into the vault.
     * Only callable by an account with the DEFAULT_ADMIN_ROLE.
     *
     * Requirements:
     * - The `assetIsImpaired()` function must return a value greater than 0.
     *
     * @param _to The address to which the impaired asset balance will be transferred.
     */
    function transferImpairedAssetBalance(address _to) external addressNotZero(_to)
    ↪  onlyRole(DEFAULT_ADMIN_ROLE) {
        uint256 _assetImpairedBalance = assetImpairedBalance();
        require(_assetImpairedBalance>0, "Asset balance is not impaired");
        IERC20 assetToken = IERC20(asset());
        assetToken.safeTransfer(_to, _assetImpairedBalance);
    }

    /**
     * @dev Transfers the impaired vault balance to a specified address.
     * This method is used when the vault's balance is considered impaired,
     * meaning it does not reflect the expected balance due to incorrect transfers out of the vault.
     * Only callable by an account with the DEFAULT_ADMIN_ROLE.
     *
     * Requirements:
     * - The `vaultImpairedBalance()` function must be return a value greater than 0.
     *
     * @param _to The address to which the impaired vault balance will be transferred.
     */
    function transferImpairedVaultBalance(address _to) external addressNotZero(_to)
    ↪  onlyRole(DEFAULT_ADMIN_ROLE) {
        uint _vaultImpairedBalance = vaultImpairedBalance();
        require(_vaultImpairedBalance>0, "Vault balance is not impaired");
        IERC20(address(this)).safeTransfer(_to, _vaultImpairedBalance);
    }
```

**Cyfrin:** Acknowledged.

## 7.4 Gas Optimization

### 7.4.1 Unnecessary modifier

**Description:** The function `redeem()` has a modifier `receiverSenderNotEqual(_owner)` and it ensures that the caller is actually the owner of the shares. But this modifier is unnecessary here because it is checked in the `ERC4626::_withdraw()` function.

```
openzeppelin-contracts-upgradeable\contracts\token\ERC20\extensions\ERC4626Upgradeable.sol
291:         ERC4626Storage storage $ = _getERC4626Storage();
292:         if (caller != owner) {
293:             _spendAllowance(owner, caller, shares);
294:         }
```

**Securitize:** Modifier deleted in 52350aa.

**Cyfrin:** Verified.